Tensor và autograd

Từ mảng nhiều chiều tới gradient tự động

Chương 3 · Bài 3.1

Mảng này mang nghĩa gì?

Hai mẫu dữ liệu, mỗi mẫu có ba đặc trưng.

Ta muốn tạo hai đặc trưng mới rồi đo lỗi dự đoán.

```
shape = (2, 3)
```

- Chiều 0: mẫu.
- Chiều 1: đặc trưng.
- Tensor còn có dtype và device.

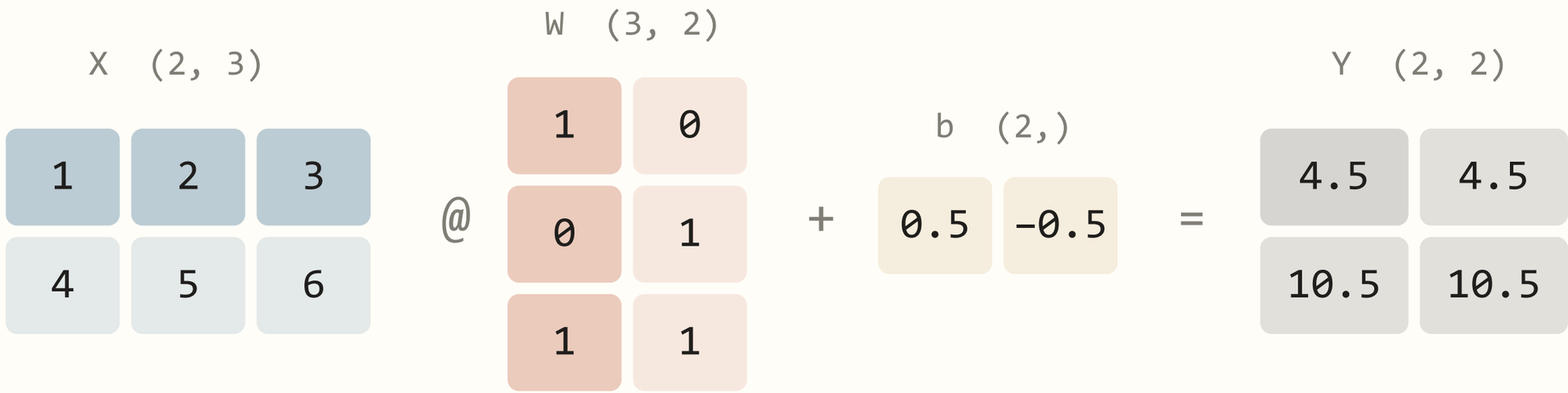
Mục tiêu

- Đoán shape của tensor trước khi chạy code.
- Phân biệt nhân từng phần tử, nhân ma trận, reshape và đổi chiều.
- Tính gradient bằng tay và kiểm bằng autograd.
- Dùng `detach`, `no_grad`; chuyển tensor giữa CPU và GPU.

01

Shape trước, code sau

Chiều nào bị gộp?



`1·1 + 2·0 + 3·1 + 0.5 = 4.5`

chiều 3 bị gộp; hai mẫu giữ nguyên

X @ W + b: chiều 3 bị gộp, hai mẫu giữ nguyên.

Shape của một lớp linear

là số mẫu, là số feature, là số feature đầu ra.

Tensor	Shape
X	(B,d)
W	(d,h)
b	(h,)
Y	(B,h)

```
X = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
W = torch.tensor([[1., 0.], [0., 1.], [1., 1.]])
b = torch.tensor([0.5, -0.5])
Y = X @ W + b
print(Y.tolist())
```

KẾT QUẢ

```
[[4.5, 4.5], [10.5, 10.5]]
```

Mỗi hàng vẫn thuộc một mẫu.

dtype đi theo dữ liệu

```
print(torch.tensor([1, 2, 3]).dtype,
      torch.tensor([1., 2., 3.]).dtype)
```

KẾT QUẢ

```
torch.int64 torch.float32
```

- Trọng số, đặc trưng: `float32`.
- Nhãn lớp, token ID: `int64` (`long`).
- Tensor số nguyên không đặt được `requires_grad=True`.

HỎI NHANH

`X` có shape `(8, 3)`, `W` có shape `(3, 5)`. `X @ W` có shape gì?

A `(3, 3)`

B `(5, 8)`

C `(8, 5)`

D `(8, 3, 5)`

Cùng shape, khác phép

`A = torch.arange(6).reshape(2, 3)`; hai hàng ban đầu là `[0,1,2]` và `[3,4,5]`.

```
print(A.reshape(3, 2).tolist())
print(A.transpose(0, 1).tolist())
```

KẾT QUẢ

```
[[0, 1], [2, 3], [4, 5]]
[[0, 3], [1, 4], [2, 5]]
```

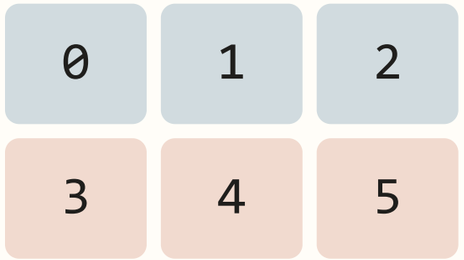
Reshape chia lại dãy phần tử.

Transpose đổi hàng và cột.

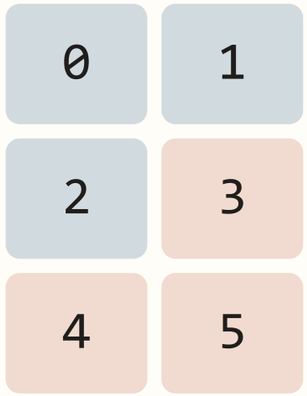
Shape đúng chưa chứng minh ý nghĩa đúng.

Reshape và transpose

`A  (2, 3)`

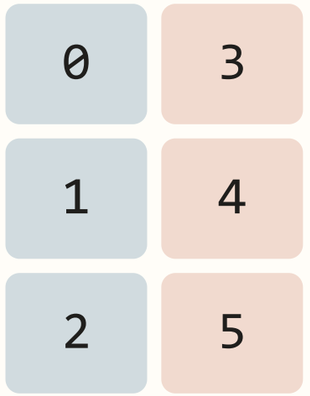


`A.reshape(3, 2)`



hàng đầu sau reshape: [0, 1]

`A.transpose(0, 1)`



hàng đầu: [0, 3]

Cùng shape (3, 2): reshape giữ thứ tự, transpose đưa cột thành hàng.

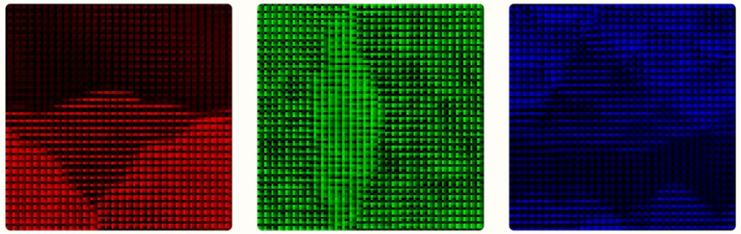
permute và reshape trên một ảnh thật



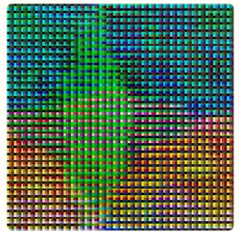
`x`: shape (96, 96, 3), kênh màu ở chiều cuối



`x.permute(2, 0, 1)`: (3, 96, 96), ba lát là đúng ba kênh R, G, B



`x.reshape(3, 96, 96)`: cũng (3, 96, 96), nhưng mỗi "kênh" là một phần ba dãy số, trộn cả R, G, B



xem lại kết quả reshape như một ảnh: hình bị xé thành sọc

Đổi (H, W, 3) sang (3, H, W): permute giữ đúng kênh màu, reshape trộn lẫn.

HỎI NHANH

Bảng `A` có shape `(2, 3)`. Muốn đổi thành `(3, 2)` sao cho các cột cũ thành hàng mới, ta dùng gì?

A `A.reshape(3, 2)`

B `A.view(3, 2)`

C `A.unsqueeze(0)`

D `A.transpose(0, 1)`

Broadcasting âm thầm

Prediction `(B,1)` trừ target `(B,)`.

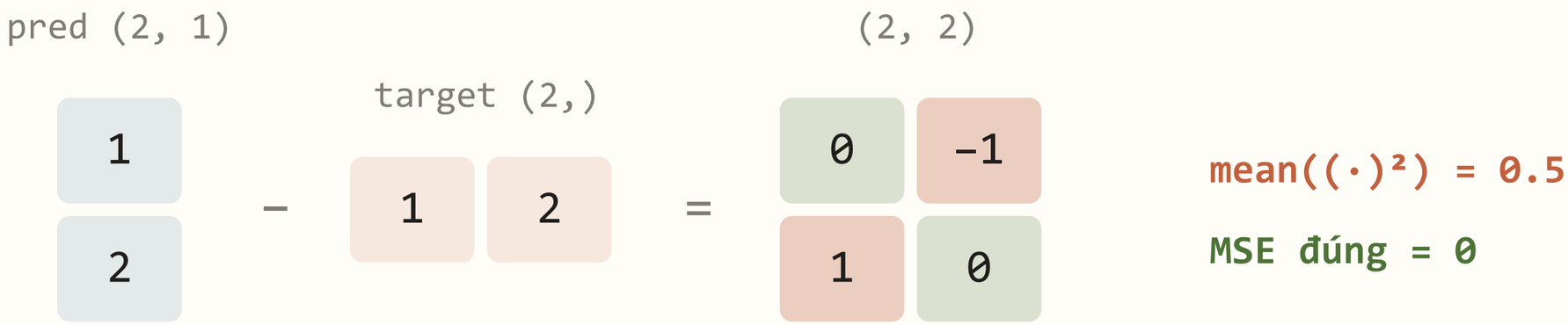
Kết quả thành `(B,B)`: so mọi cặp mẫu.

Sửa: `pred.squeeze(-1)`.

Chặn sớm:

```
assert pred.shape == target.shape
```

(2, 1) trừ (2,)



mỗi dự đoán bị trừ cho mọi nhãn; chỉ đường chéo là cặp đúng

(2, 1) trừ (2,) thành bảng (2, 2), nên MSE ra 0.5 thay vì 0.

HỎI NHANH

Dự đoán `pred = [[1.], [2.]]`, shape `(2, 1)`, khớp đúng nhãn `target = [1., 2.]`, shape `(2,)`. Vậy mà `((pred - target) ** 2).mean()` lại in ra `0.5`. Vì sao?

A `float32` làm tròn 1 và 2 nên hiệu khác 0

B Phép trừ bị broadcast thành `(2, 2)`, so mọi cặp với nhau

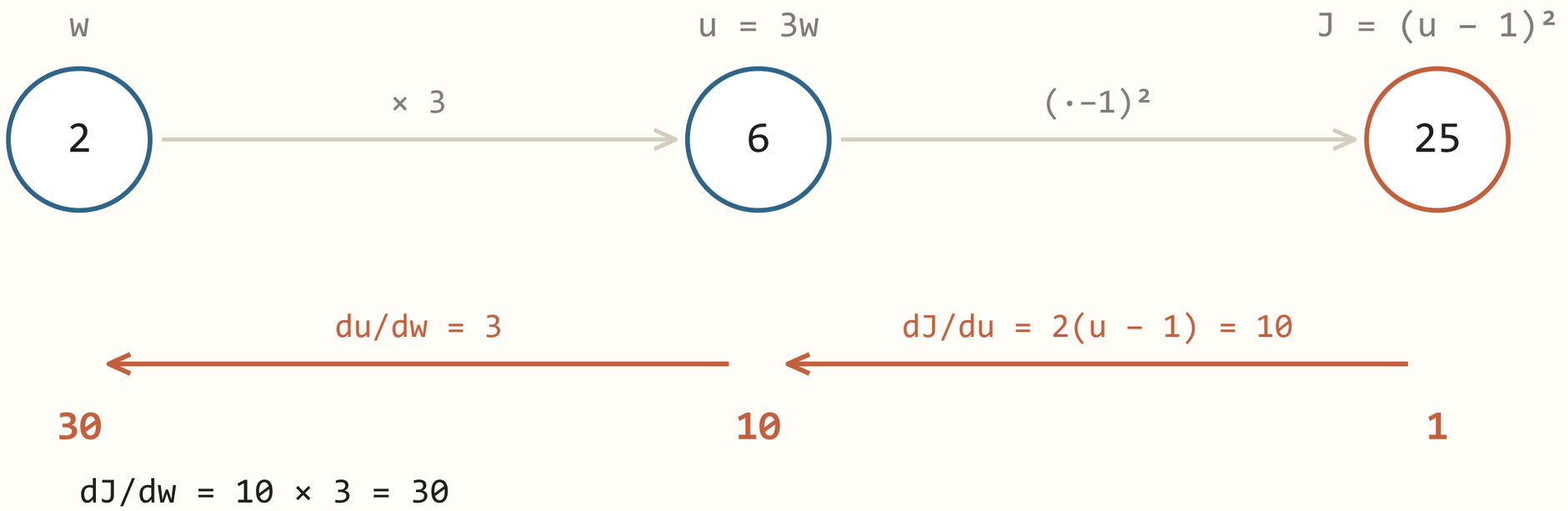
C Nhãn phải có kiểu `long` thì phép trừ mới đúng

D `mean()` chia thêm cho chiều thứ hai

02

Gradient đi ngược DAG

Một đường tính toán nhỏ



Forward 2 → 6 → 25; backward nhân đạo hàm cục bộ: 10 × 3 = 30.

Đồ thị dựng trong lúc chạy

Forward chạy đến đâu, DAG được ghi đến đó; mỗi forward dựng lại DAG mới.

`if` và vòng lặp Python vẫn đạo hàm được: backward theo nhánh **đã chạy**.

Một biến đi qua nhiều nhánh: gradient các nhánh được **cộng**.

HỎI NHANH

Cho `x = torch.tensor(-3., requires_grad=True)`, rồi `y = 2 * x if x > 0 else x ** 2` và `y.backward()`. `x.grad` bằng bao nhiêu?

A `-6.0`, đạo hàm của nhánh `x ** 2`

B `2.0`, đạo hàm của nhánh `2 * x`

C `-2.0`, trung bình đạo hàm của hai nhánh

D Báo lỗi, vì graph phải cố định trước khi forward

Đạo hàm qua hai bước

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Gradient dương: tăng x một chút thì loss tăng.

Autograd đối chiếu

`requires_grad=True`: ghi lại mọi phép tính dùng `w`.

```
w = torch.tensor(2., requires_grad=True)
u = 3 * w
J = (u - 1) ** 2
J.backward()
```

Kết quả trong notebook:

KẾT QUẢ

```
25.0 30.0
```

Loss 25 · gradient 30.

Backward chưa cập nhật trọng số.

Bước cập nhật là việc riêng

`with torch.no_grad():` khi cập nhật thủ công.

Optimizer sẽ làm bước ấy trong training loop.

HỎI NHANH

Vẫn , nhưng tại . Gradient bằng bao nhiêu, và một bước gradient descent đẩy theo hướng nào?

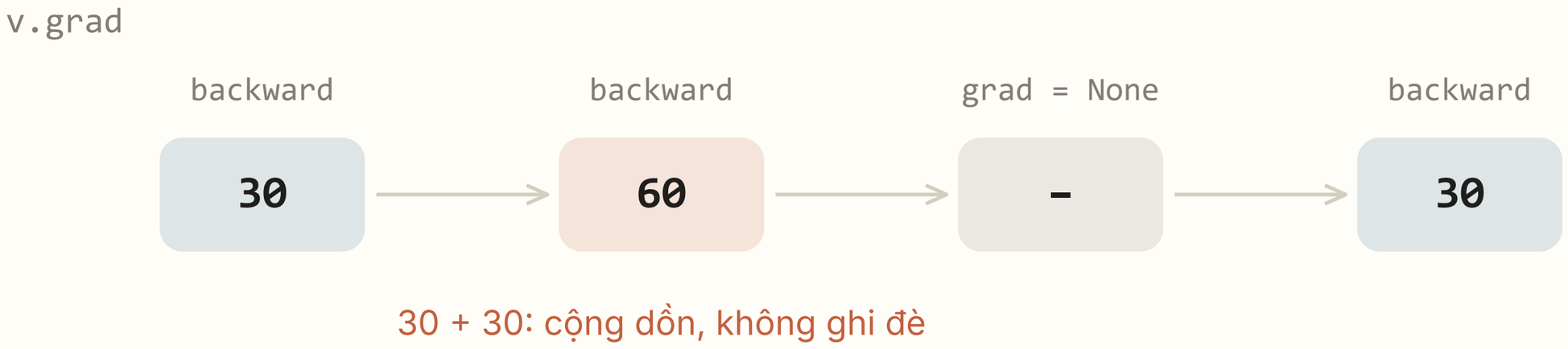
A , giảm , tức đi ngược dấu gradient

B , tăng , tức đi theo dấu gradient

C , giảm , tức đi theo dấu gradient

D , tăng , tức đi ngược dấu gradient

Mỗi lần backward cộng thêm



30 + 30: cộng dồn, không ghi đè

Mỗi lần backward cộng thêm: 30 rồi 60; xóa grad thì lại 30.

Gradient cộng dồn

Hai lần forward/backward độc lập.

Giữa lần hai và ba: `v.grad = None`.

KẾT QUẢ

```
30.0
60.0
30.0
```

Xóa gradient cũ trước bước mới.

Mỗi bước: forward mới → backward mới.

HỎI NHANH

Một bạn tính `loss` một lần trước vòng lặp, rồi mỗi epoch gọi `loss.backward()` và `optimizer.step()`. Sang epoch thứ hai thì báo `RuntimeError: Trying to backward through the graph a second time`. Sửa thế nào?

A Thêm `retain_graph=True` để graph không bị giải phóng

B Gọi `optimizer.zero_grad()` trước lần `backward()` thứ hai

C Đưa forward và phép tính `loss` vào trong vòng lặp

D Đổi thành `loss.detach().backward()` để tách khỏi graph cũ

03

Graph và thiết bị

Ba thao tác, ba mục đích

- `detach()`: cắt đường gradient.
- `clone()`: sao dữ liệu.
- `.item()`: lấy số Python để ghi log.
- `no_grad()`: ngừng ghi graph trong khối.

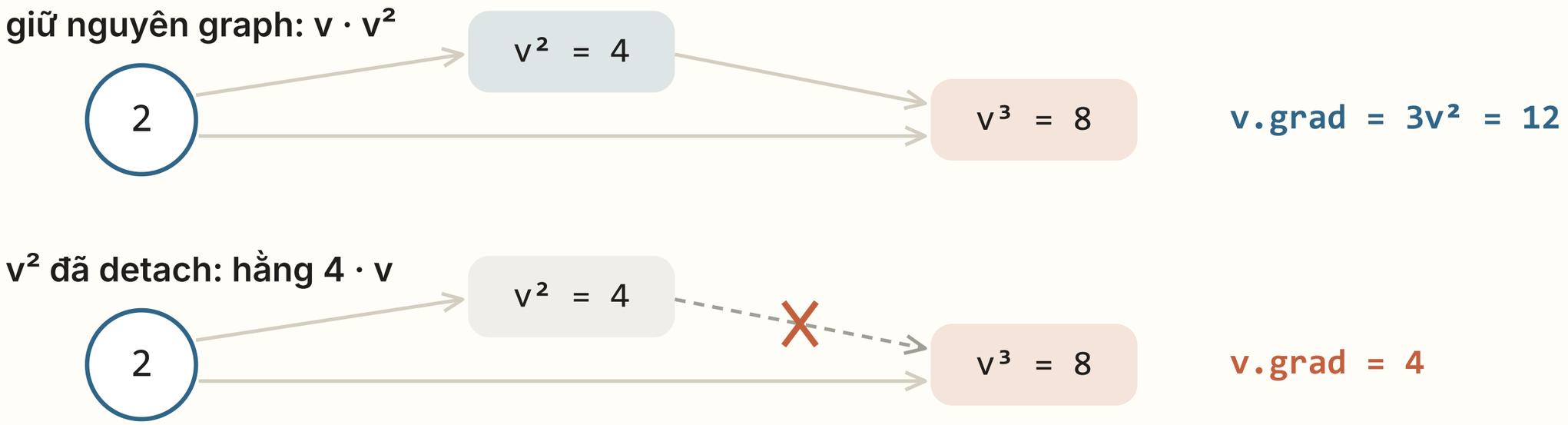
Tại v=2:

```
v.square().detach() * v
```

Forward là 8, gradient là **4**.

Nhánh square bị coi là hằng số.

Detach cắt một đường



Giữ graph: gradient 12. Detach v²: chỉ còn 4.

HỎI NHANH

`w` bằng 2 và đã có `w.grad` bằng 30. Chạy `snap = w.detach()`, rồi `with torch.no_grad(): w -= 0.1 * w.grad`. `snap.item()` in ra gì?

A `2.0`, vì `snap` đã tách khỏi graph

B `2.0`, vì `no_grad` không làm đổi giá trị của `w`

C `-1.0`, vì `snap` dùng chung vùng nhớ với `w`

D Báo lỗi, vì tensor đã detach thì không đọc được

Cùng device mới tính cùng nhau

```
device = torch.device(
    'cuda' if torch.cuda.is_available()
    else 'cpu')
X_device = X.to(device)
linear = torch.nn.Linear(3, 2).to(device)
```

Tensor: `.to` trả bản mới, phải gán lại.

Module: `.to` chuyển tại chỗ.

Máy chạy bài không có CUDA:

KẾT QUẢ

```
cpu (2, 2)
(2, 2)
```

Có CUDA thì cùng code chọn `cuda`.

GPU không tự nhanh hơn với dữ liệu rất nhỏ.

HỎI NHANH

Model đã gọi `model.to('cuda')`. Trong vòng lặp có dòng `X.to('cuda')`, không gán lại, rồi `model(X)`, và PyTorch báo các tensor nằm trên hai device khác nhau. Vì sao?

A `X.to` trả về một bản mới, còn `X` cũ vẫn ở CPU

B `model.to` cũng phải gán lại thì model mới chuyển

C Phải dùng `X.cuda()`, vì `.to` chỉ đổi dtype

D Batch quá nhỏ nên PyTorch giữ `X` ở CPU

Gradient bằng None hay sai số?

Lần theo đường bị cắt:

- `detach()`, `no_grad()`
- `.item()`, `float()` giữa forward
- tham số không tham gia forward

```
u = torch.tensor(3 * w.item())
J = (u - 1) ** 2 + w
```

Đúng ra ; autograd chỉ thấy phần , cho 1.

Khi đi thi

- `assert pred.shape == target.shape`.
- Nhãn lớp `long`; hồi quy, BCE `float`.
- `device` đặt một chỗ, không gõ cứng `.cuda()`.

04

Luyện đề

LUYỆN ĐỀ 1

Một model khai báo ba lớp `fc1`, `fc2`, `fc3` trong `__init__`, nhưng `forward` chỉ gọi `fc1` rồi `fc2`. Sau `loss.backward()`, `fc3.weight.grad` là gì?

A Tensor toàn 0, vì `fc3` không ảnh hưởng tới loss

B `None`, vì `fc3` không có trong đồ thị của forward

C Gradient bình thường, vì `fc3` có trong `model.parameters()`

D Báo lỗi lúc backward, vì có tham số không được dùng

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 54

LUYỆN ĐỀ 2

`X` có shape `(64, 10)`: 64 mẫu, mỗi mẫu 10 đặc trưng. Để trừ mỗi hàng cho trung bình của chính hàng đó, một bạn viết `m = X.mean(dim=1)` rồi `X - m`. Kết quả thế nào?

A Báo lỗi, vì `(64, 10)` và `(64,)` không broadcast được

B Chạy đúng, mỗi hàng trừ trung bình của chính nó

C Chạy được, nhưng ra `(64, 64)`, so mọi cặp mẫu

D Chạy được, nhưng trừ trung bình theo từng cột

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 104

LUYỆN ĐỀ 3

`loss` là tensor 0 chiều, `logits` là tensor `(32, 10)`, cả hai còn nối với graph. Ta muốn (a) ghi loss vào log dưới dạng số Python, và (b) giữ `logits` ở dạng tensor nhưng không giữ graph. Gọi gì?

A (a) `loss.detach()`, (b) `logits.detach()`

B (a) `loss.detach()`, (b) `logits.item()`

C (a) `loss.item()`, (b) `logits.item()`

D (a) `loss.item()`, (b) `logits.detach()`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 145

Chốt ý

Tensor có số, shape, dtype và device; đọc cả bốn trước khi tính.

Autograd ghi lại phép tính lúc forward; backward cộng gradient theo mọi đường, chưa cập nhật trọng số.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: trước mỗi ô, đoán shape, dtype và gradient sẽ in ra, rồi đối chiếu.